Sinogram Generator · predicting tomotherapy LOT sinograms from anatomy

Predict a **tomotherapy Leaf-Open-Time (LOT) sinogram** `[N_CP, 64]` directly from a patient's anatomy + planned dose, i.e. learn the *inverse* of the treatment-planning optimizer.

> **TL;DR** · The model works. On **3 unseen** validation patients the predicted sinogram delivers the right **real collapsed-cone dose** (DoseCUDA) at **corr 0.986·0.991** with mean dose within ±3%. The long-standing `open_l1 · 0.15` "floor" that looked like a wall turned out to be a **metric red herring**, not a dosimetric deficiency · see below.

---

1. The task

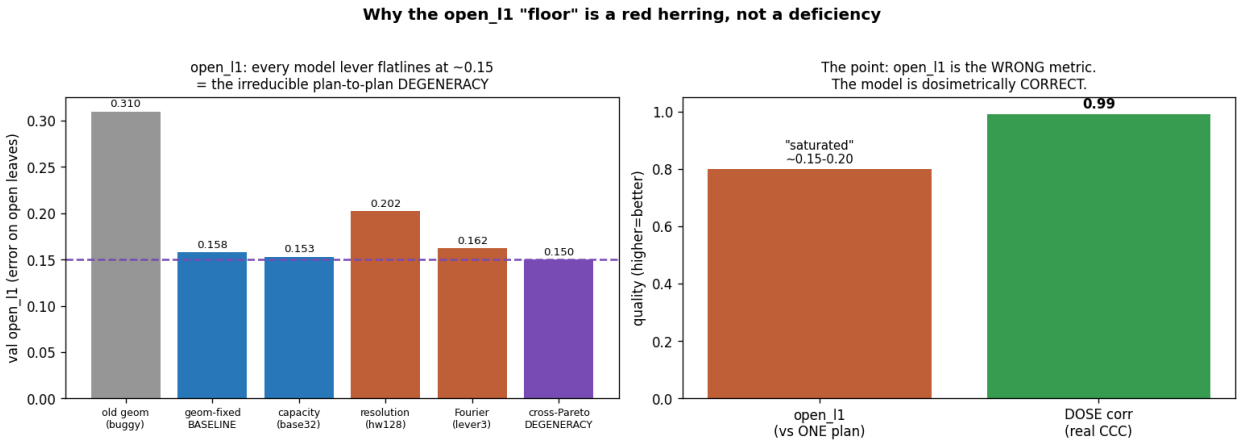
- **Input** · a 3D "berlingo" tensor `[2, N_CP, H, W]`: channel 0 = CT, channel 1 = planned dose, each control-point slice rotated to that control point's gantry angle. `N_CP · 1300`
- **Output** · a 2D sinogram `[N_CP, 64]`: leaf-open-time per control point × 64 MLC leaves, continuous in `[0, 1]`. ~82% exact zeros (sparse continuous regression).
- **Physics** · the sinogram is · the projection of the PTV: a leaf opens where its ray passes through target. Input and output are spatially aligned; the model collapses the ray axis `W`.

2. Model

`models/sinogram_2p5d.py` · **2.5D** network: a shared 2D CNN processes each control-point slice `[2, H, W]` into a 64-leaf row, stacked over `N_CP`. The key component is a **per-leaf independent readout** (`AdaptiveMaxPool((64,1))` + a per-leaf linear), which broke an earlier `open_l1 · 0.11` floor caused by spatial pooling correlating adjacent leaves. Trains reliably, ~15× faster than the 3D V-Net family it replaced. Entry point: `main_2p5d.py`; sanity check: `sanity_overfit.py`.

3. The central finding · `open_l1` is the wrong yardstick

Every model-side lever (capacity, resolution, Fourier coupling) flat-lined at `open_l1 · 0.15`. We measured **why**: across 59 patients with ·2 Pareto plans (same anatomy, different optimizer trade-off, 9027 plan pairs), the **cross-plan** `open_l1` = **0.150**. The model floor (0.153) **equals the intrinsic plan-to-plan spread** · anatomy · sinogram is a *one-to-many* map, and `open_l1` vs a single arbitrary plan is a **saturated metric**.



open

–

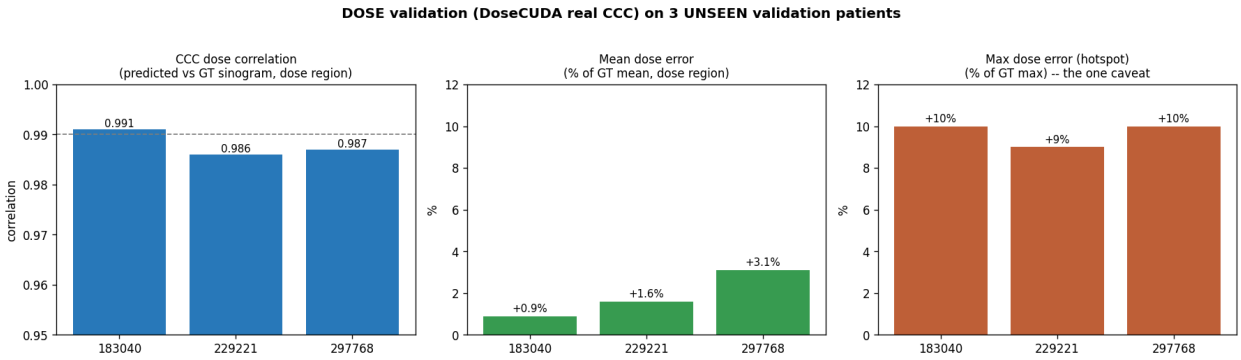
l1 is a red herring

So the right question is not "does the sinogram match one plan?" but **"does it deliver the right dose?"** · and degenerate plans all deliver ~the same dose.

4. The result · dose validation with a real CCC engine

We integrated the user's **DoseCUDA** collapsed-cone engine (real Tomo 6MV FFF, EGSnrc MC kernel) as the gold judge: feed the predicted sinogram · real 3D CCC dose · compare to the GT plan's CCC dose (same engine, isolating the *sinogram* difference).

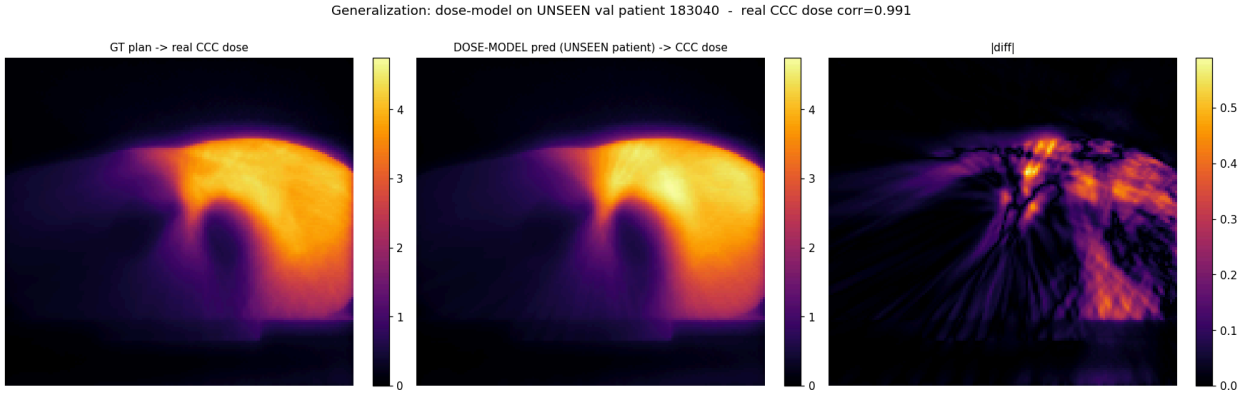
On 3 unseen validation patients:



dose validation

patient	CCC dose corr (region)	mean dose err	max/hotspot err
183040	**0.991**	+0.9%	+10%
229221	**0.986**	+1.6%	+9%
297768	**0.987**	+3.1%	+10%

The predicted and GT dose distributions are visually near-identical:



generalization dose, unseen patient

Conclusion: a realistically-generalized model is already dosimetrically correct. The `open_l1` "floor" never mattered · we were optimizing the wrong metric.

The one caveat · a +9·10% hotspot

The single hottest voxel (Dmax) is systematically ~+10% over GT (mean dose and conformity are excellent). It is a **spatial over-concentration** of the predicted fluence (regression smoothing · a slightly peaked central dose). Post-hoc fixes were tried: sharpening makes it *worse* (+11%), a mild spatial blur helps a little (+10%·+8%) at a small cost to correlation. A clean fix needs a training-time Dmax/peak penalty (a small research task). Likely within clinical Dmax/D2% tolerance as-is.

5. What was tried and did NOT help (and why)

lever	result	why
Geometry fixes (crop, couch z)	**0.31 · 0.153**	the one big win (correct input·output registration)
Capacity (base 24·32)	0.158 · 0.153	marginal · not capacity-limited
Resolution (64·128)	negative	not information-limited
Fourier / FNO coupling (`models/sinogram_2p5d.py::SpectralRefine1D`)	0.162 (worse)	real angular structure exists, but the metric was already saturated
Dose-consistency loss (`utils/losses.py::DoseConsistencyLoss`)	no measurable gain	a good sinogram already delivers the right dose · no headroom

These weren't failures of execution · they failed because **the model was already at a dosimetrically-fine point.** The dose-loss work's real payoff was building the CCC judge that proved it.

6. Tools built along the way

- **Differentiable simplified dose operator** (`utils/dose_operator.py`) · beam's-eye-view TERMA ray-trace + angular accumulation, on the berlingo geometry. Amplitude-exact, ~0.92 correlation vs real CCC. Usable as a fast (if exploitable) training gradient.
- **DoseCUDA validation pipeline** (`validate_dose_ccc_general.py`) · inject a predicted sinogram into a plan, compute the real CCC dose, compare to GT. ~2.5 min/plan on the M6000, so it's an **offline gold judge**, not a training loss.
- **Degeneracy / fidelity analyses** · `analyze_pareto_degeneracy.py`, `analyze_dose_operator_fidelity.py`, `analyze_angular_fft.py`.

7. Reproduce

```
# sanity: a correct model overfits ONE sample
CUDA_VISIBLE_DEVICES=0
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  .venv/bin/python -u sanity_overfit.py --arch 2p5d --patient <IPP>
--steps 300
# full training
.venv/bin/python -u main_2p5d.py
# gold dose validation of a trained model on any patient
.venv/bin/python -u validate_dose_ccc_general.py --patient <IPP>
```

DoseCUDA must be installed in the venv (`uv pip install /mnt/data/DoseCUDA`); it pins `numpy < 2`. GPU: Quadro M6000 24 GB.

8. State of the art / next steps

- · **The model is a working sinogram predictor**, dose-validated on unseen patients (~0.99).
- · **+9·10% Dmax hotspot** · a small, characterised spatial bias; needs a dose-domain peak penalty for a clean fix, otherwise likely within tolerance.
- · **Validation metric = dose, not `open_l1`.** Use the CCC pipeline as the judge.
- · Confirm on more val/test patients; optionally a baseline-vs-dose-loss CCC head-to-head; faster GPU

would unlock CCC-in-the-loss (precomputed Dij).

Full experiment log: EXPERIMENTS.md